

[RedTeam 3차 프로젝트 코드 계획]

- main코드에서 항목별로 선택해서 진행하는 방식
- main 메뉴 : File열람 / 1. 정보수집 / 2. 스캐닝 / 3. 취약점분석 / 4. 공격
- 기본적으로 저장되어 있는 json file에서 분석/공격 대상에 대한 정보를 가져오는 걸로 진행

* 각 항목별로 어떻게 코드를 구성해야 할지 초안이 적혀 있음.
 * 추후 함수명은 추가 예정.
 * 함수명과 설명이 이미 적혀있는 것들은 2차에서 만들었던 코드를 분석해 둔 내용. 3차에서 그대로 사용할건 아니라서 참고만 해주기.

+ 더 정해야 하는 것들

- 저장 파일 이름 및 구성
- 함수명/클래스명
- 데이터 연결성 고민

File 열람

정보수집/스캐닝/취약점분석/공격 에 대한 결과 json/txt 파일들을 볼 수 있음
 더 자세한 내용은 차차 추가할 예정

1. 정보수집 (Gatherinfo)

- 1) 정보수집 대상 : ip/도메인
 - [input] : 직접 입력
 - [.txt file] : ip/도메인 list txt파일을 읽고 출력 후 '특정 번호' or '전체선택' 입력받아서 실행

>> 2가지 방법 다 넣기
 >> 추가적으로 입력 받아야 하는 것들은 전부 file 만들기!
- 2) 옵션 : 사용하고자 하는 상세 기능
 - [list] : 옵션을 list화 하여 목록 출력 후 번호로 입력받게 하기
 - [yes/no] : 선택지로 받기

>> 둘 중 적절한걸로 선택해서 하기
- 3) 실행 결과 : 나오는 결과값 파일로 저장
 - 결과 print 로 출력
 - 출력된 결과 중 ip/도메인 등 필요한 정보들만 json파일(dict구조)로 저장

>> 저장된 파일은 추후 '취약점분석' 및 '공격'에 이용

기술		함수명	설명	return
dig				
dnsrecon				
dnsenum				

2. 스캐닝

1) 정보수집 대상 : ip/도메인 or 대역

- [input] : 직접 입력
- [.txt file] : ip/도메인 list txt파일을 읽고 출력 후 '특정 번호' or '전체선택' 입력받아서 실행
- [.json file] : '1.정보수집'에서의 .json파일을 불러와서 ip list 출력 후 '특정 번호' or '전체선택' 입력받아서 실행

>> [input] 이랑 [file] 둘 다 넣는데 .json/.txt는 각자 알아서 적절한걸로 넣기

>> 추가적으로 입력 받아야 하는 것들은 전부 file 만들기!

2) 옵션 : 사용하고자 하는 상세 기능

- [list] : 옵션을 list화 하여 목록 출력 후 번호로 입력받게 하기
- [yes/no] : 선택지로 받기

>> 둘 중 적절한걸로 선택해서 하기

3) 실행 결과 : 나오는 결과값 파일로 저장

- 결과 print 로 출력
- 출력된 결과 중 ip/도메인 등 필요한 정보들만 json파일(dict구조)로 저장

>> 저장된 파일은 추후 '취약점분석' 및 '공격'에 이용

기술		함수명	설명	return
nmap	0	nmap_start()	main 실행부 [4, 5, 6] 정보 수집 및 조립 [input] 원격서버 실행 여부 → ! 사용자 정보 직접 입력 및 수정!! → ! 계정 비번도 직접 수정!! → 결과 출력	
	1	print_nmap_menu (menu: dict) -> None	NMAP_MENU 출력	
	2	parse_targets (raw_target: str)	(raw_target) 타겟 파싱	타겟 리스트
	3	parse_ports (raw_port: str) -> str	(raw_port) 포트 파싱	포트
	4	collect_target_and_port()	[input] 타겟 ip/범위/파일명 → [2] 파싱 [input] 포트 → [3] 파싱	타겟정 보
	5	select_nmap_options (menu: dict) -> list	[1] nmap 옵션 출력 [input] 옵션 선택	옵션
	6	build_final_command ()	(flags: list, target_kind: str, target_value, port_token: str) -> str command 명령어 조립	최종 명령어
arp-scan				

3. 취약점 분석(VulnerAnalysis) - Injection

*injection을 넣을거면 이 부분은 추후 더욱 보강해야함

1) 분석 대상 : url / 파라미터

- [input] : 직접 입력
- [.json file] : '1','2' 번에서의 .json파일을 불러와서 기술에 필요한 정보만 출력 후 특정 번호를 입력 받아서 실행
- [.txt file] : url / 파라미터 list txt파일을 읽고 출력 후 특정 번호를 입력 받아서 실행

>> [input] 이랑 [file] 둘 다 넣는데 .json/.txt는 각자 알아서 적절한걸로 넣기

>> 추가적으로 입력 받아야 하는 것들은 [input] or [file] 만들기!

2) 옵션 : X 없음.

- SQL Injection / Command Injection

>> injection 둘 다 돌리기

3) 실행 결과 : 나오는 결과값 파일로 저장

- 결과 print 로 출력
- SQL/Command 중 해당하는 취약점이 있으면
 - > [file]에 있던 '대상'에 취약점분석을 시도했을 경우 → 기존 파일에 이어서 '취약점 여부' 필드만 추가 후 저장
 - > [input]으로 받은 대상일 경우 → 새로운 json파일 생성

>> 저장된 파일은 추후 '공격'에 이용

기술		함수명	설명	return
SQLInjection class SQLInjection()	0	main()	main 실행부 [1,2] 진행할 서비스 목록 출력 [input] 서비스 번호 → [4,5,6,7,12]	
	1	__init__()	생성자 메서드	
	2	preprocess()	url, param, cookie를 초기화 하는 메서드 [input] 공격할 url, 쿠키값	
	3	checkInjection()	Injection 취약 여부 사이트인지 확인	
	4	getDBs()	DB 목록을 탈취하는 메서드 → [8] 결과값 출력	
	5	getTables (self, dbName)	특정 DB의 Table 목록을 탈취하는 메서드 → [8] 결과값 출력	
	6	getColumns	(self, dbName, tableName) 특정 Table의 열 목록을 탈취하는 메서드 → [8] 결과값 출력	
	7	getDatas	(self, dbName, tableName, column) 특정 Table의 데이터 값을 탈취하는 메서드 → [8] 결과값 출력	

	8	printInformation (self, info, typ)	탈취한 정보 값을 화면에 출력 [input] 저장 y/n → [11] 결과 저장	
	9	requestGET()	GET 요청 메서드	
	10	basicSQL (self, sql)	Blind Injection을 제외한 나머지 SQL Injection 기법으로 데이터를 추출	
	11	saveData (self, data)	추출된 데이터 파일 저장	
	12	decryption()	데이터 복호화 메서드 [input] 파일 경로 → 복호화한 데이터 출력	
Command Injection class CommandInje ction()	0	main()	전반적인 코드가 실행되는 메서드 [input] 취약점분석/공격 선택 → [3, 4] 취약점분석/공격 실행	
	1	__init__()	필수 속성들 선언 및 초기화 메서드 main 메서드 실행	
	2	preprocess()	url 및 쿠키 값 초기화 [input] 공격 url, 쿠키값, 공격 코드 파일	
	3	gatherInfo()	ping 테스트 및 명령 실행 가능 여부 등 확인 [6] 요청, [5] 결과 데이터 저장	
	4	injection()	[6] 요청, [5] 결과 데이터 저장	
	5	saveInfo (self, flag, data)	탈취한 정보를 파일로 저장	
	6	requestData (self, insertData)	POST 요청 및 응답 값 반환	

4. 공격(Attack)

- 1) 공격 대상 : ip / port / url
 - [input] : 직접 입력
 - [.json file] : '1','2','3' 번에서의 .json파일을 불러와서 각 기술마다 필요한 정보만 출력 후 특정 번호를 입력 받아서 실행
 - [.txt file] : 각 기술마다 추가로 공격에 필요한 파일들

>> 되도록이면 [json file]의 대상 정보를 이용하여 공격 대상 선정하기
 >> 추가적으로 입력 받아야 하는 것들은 [input] or [file] 만들기!
- 2) 옵션 : 사용하고자 하는 상세 기능
 - [list] : 옵션을 list화 하여 목록 출력 후 번호로 입력받게 하기
 - [yes/no] : 선택지로 받기

>> 둘 중 적절한걸로 선택해서 하기
- 3) 실행 결과 : 공격 성공 여부 확인 및 최종 저장
 - 결과 print 로 출력
 - 공격 성공여부 / 획득한 정보 등 json 및 txt 파일로 저장
 - 저장 목적 :
 - > json : 데이터형식으로 보관하여 사용 가능할 수 있게
 - > txt : 보고서용

기술		함수명	설명	return
arp-spoofing				
msf-venom				
scapy				
hydra	1	__init__()	"password.txt" 필요	
class hydramanager ()	2	start_attack()	hydra 전체적인 메인 함수 [input] 타겟 ip, 서비스(?), 이용방식 입력 → [input] or [파일] 중 선택하여 입력 → 공격 실행. 결과값 출력	
request			ip, domain, login정보 코드에 직접 입력 → get, post 코드로 직접 수정 후 실행	